
UD01 – Actividades voluntarias y obligatoria:

Introducción a Python



Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro



GENERALITAT VALENCIANA
CONSELLERIA D'EDUCACIÓ, INVESTIGACIÓ, CULTURA I ESPORT

Autor: Óscar García Escudero
Asignatura: ADA

Índice

Introducción.....	4
Objetivos.....	4
Desarrollo de las actividades.....	5
Actividad UD01.....	5
Resultado.....	6
Actividad UD02.....	6
Resultado.....	8
Actividad UD03.....	9
Resultado.....	12
Actividad UD04.....	14
Resultado.....	16
Actividad Obligatoria.....	18
Resultado.....	22
Problemas encontrados, soluciones y/o sugerencias.....	24
Conclusión.....	24
Bibliografía/webgrafía.....	24

Licencia

**Reconeixement - No comercial – CompartirIgual(BY-NC-SA):**

No es permet un ús comercial de l'obra original ni de les possibles obres derivades, la distribució de les quals s'ha de fer amb una llicència igual a la que regula l'obra original.

Introducción

En esta unidad didáctica se va a realizar un repaso paso general de los conceptos básicos necesarios para trabajar con Python. A partir de este repaso se realiza una serie de actividades voluntarias y otra obligatoria donde desarrollar de manera práctica los conceptos generales de este lenguaje.

Objetivos

- Repasar conceptos básicos de programación con python como son funciones, módulos, estructuras de control, tipos de datos complejos y excepciones.

Desarrollo de las actividades

Repositorio de GitHub compartido: <https://github.com/oscgaresc/ADA>

Actividad UD01

Escribe un programa que pida al usuario un número entero del 1 al 10 y muestre por pantalla su tabla de multiplicar.

Deberás convertir el texto introducido por el usuario al tipo entero (int) y validar que se encuentre en el rango correcto. Si no es así, mostraremos un error al usuario y acabará el programa.

```
lliurex@lliurex-VirtualBox:/media/sf_COMPARTIDA/MPAZ$ python3.8 ud2_ejer3_multiplicando.py

Dame un número entero del 1 al 10: 5

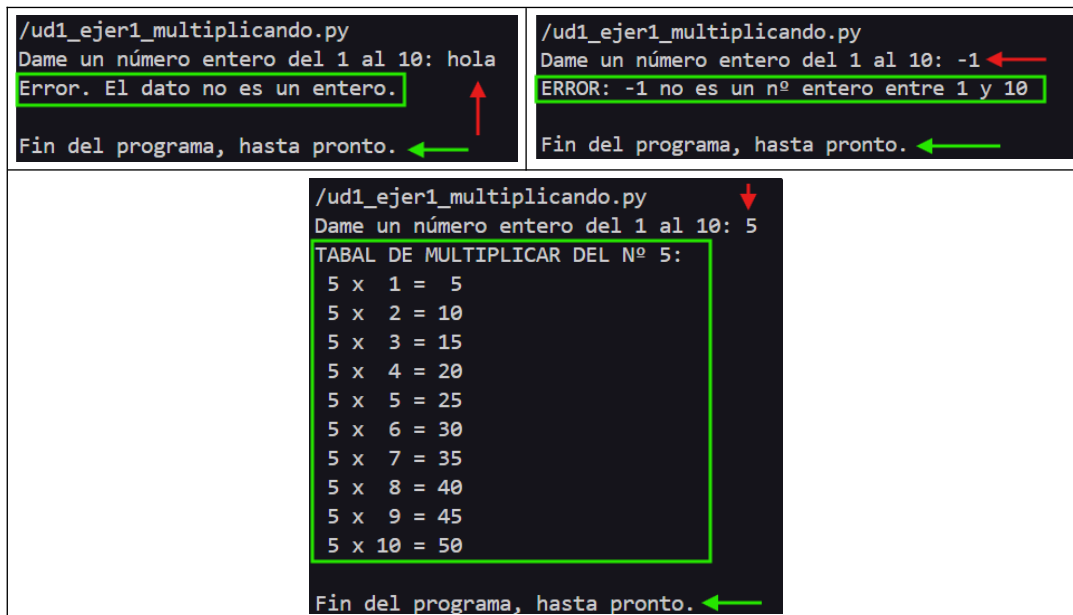
TABLA DE MULTIPLICAR DEL Nº 5 :
1 * 5 = 5
2 * 5 = 10
3 * 5 = 15
4 * 5 = 20
5 * 5 = 25
6 * 5 = 30
7 * 5 = 35
8 * 5 = 40
9 * 5 = 45
10 * 5 = 50
```

Figura 1: Ejemplo de ejecución

Se añade el código del ejercicio:

```
#try-exception para controlar el mensaje de error en caso de no añadir un entero.
try:
    # Se solicita al usuario un número entero entre 1 y 10
    entero = int(input("Dame un número entero del 1 al 10: "))
    if entero>0 and entero<=10:
        print(f'\nTABLA DE MULTIPLICAR DEL Nº {entero}:')
        for i in range(10):
            #print(f'{entero} x {i+1} = {entero*(i+1)}')
            print('\t%2d x %2d = %2d' %(entero,i+1, entero*(i+1)))
        else:
            print(f'\u001B[31mERROR: {entero} no es un nº entero entre 1 y 10')
except ValueError:
    print("\u001B[31mError. El dato no es un entero.")
finally:
    print("\n\u001B[0mFin del programa, hasta pronto.") # reseteando el color rojo
```

Resultado



The image shows three screenshots of a terminal window running a Python script named `/ud1_ejer1_multiplicando.py`. The first screenshot shows the prompt "Dame un número entero del 1 al 10:" followed by the input "hola". The second screenshot shows the input "-1" and a red error message "ERROR: -1 no es un nº entero entre 1 y 10". The third screenshot shows the input "5" and a multiplication table for the number 5, followed by the prompt "Fin del programa, hasta pronto.".

```
/ud1_ejer1_multiplicando.py
Dame un número entero del 1 al 10: hola
Error. El dato no es un entero.
Fin del programa, hasta pronto.

/ud1_ejer1_multiplicando.py
Dame un número entero del 1 al 10: -1
ERROR: -1 no es un nº entero entre 1 y 10
Fin del programa, hasta pronto.

/ud1_ejer1_multiplicando.py
Dame un número entero del 1 al 10: 5
TABAL DE MULTIPLICAR DEL Nº 5:
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
Fin del programa, hasta pronto.
```

Se ha añadido en el código que los mensajes de error aparezcan de color rojo, aunque en las imágenes no aparezca ya que se realizaron antes de modificarlo.

Actividad UD02

Escribe un programa en Python que contenga una función llamada `valida_opcion()`. Esta función mostrará un menú por pantalla y validará que la opción elegida por el usuario es una de las correctas.

Para ello, habrá que tener en cuenta los siguientes puntos:

- *La función no recibirá ningún parámetro.*
- *Estará mostrando el menú y pidiendo la opción al usuario continuamente hasta que se asegure de que la opción elegida está dentro de las correctas.*
- *Devolverá la opción elegida en la llamada.*
- *Documenta la función con un docstring donde expliques para qué sirve y cómo se utiliza.*

El menú que debe mostrar es el que aparece a continuación:

```

=====
GAMIFICACIÓN EN EL AULA
=====
1 - Cargar datos del fichero
2 - Imprimir datos
3 - Jugar
4 - Guardar datos
5 - Cambiar contraseña
0 - SALIR
-----
Dame la opción:

```

En la imagen siguiente podemos ver algunos ejemplos de ejecución del programa:

```

=====
GAMIFICACIÓN EN EL AULA
=====
1 - Cargar datos del fichero
2 - Imprimir datos
3 - Jugar
4 - Guardar datos
5 - Cambiar contraseña
0 - SALIR
-----
Dame la opción: 8
Por favor, vuelve a intentarlo.

```

Figura 2: Opción no válida

```

=====
GAMIFICACIÓN EN EL AULA
=====
1 - Cargar datos del fichero
2 - Imprimir datos
3 - Jugar
4 - Guardar datos
5 - Cambiar contraseña
0 - SALIR
-----
Dame la opción: 2
La opción seleccionada es: 2

```

Figura 3: Opción válida

A continuación se añade el código del ejercicio realizado:

```

# Función que muestra un menú de opciones que el usuario debe seleccionar
def valida_opcion():
    opcion = -1
    # Bucle cíclico para evitar que salga del menú hasta que no seleccione
    # la opción correcta
    while opcion!=0:
        print('\n\u001B[0m=====') # Resetea colores
        print('\tGAMIFICA EN EL AULA')
        print('=====')
        print('1 - Cargar datos del fichero')
        print('2 - Imprimir datos')
        print('3 - Jugar')
        print('4 - Guardar datos')
        print('5 - Cambiar contraseña')
        print('0 - Salir')
        # bucle try-exception para controlar el mensaje de error cuando
        # el usuario no añada un entero

```

```

try:
    opcion = int(input('Indica la opción seleccionada (0-5): '))
    #bucle condicional para seleccionar una de las opciones del menú
    if opcion == 1:
        print(f'La opción seleccionada es: {opcion}')
    elif opcion == 2 :
        print(f'La opción seleccionada es: {opcion}')
    elif opcion == 3 :
        print(f'La opción seleccionada es: {opcion}')
    elif opcion == 4 :
        print(f'La opción seleccionada es: {opcion}')
    elif opcion == 5 :
        print(f'La opción seleccionada es: {opcion}')
    elif opcion == 0 :
        print(f'\nFin del programa, hasta pronto.')
    else:
        print(f'\u001B[31mERROR: {opcion} no es un nº entero entre 0 y 5, vuelve a intentarlo.')

except ValueError:
    print("\u001B[31mError. El dato no es un entero, vuelve a intentarlo.")
    opcion = -1
# Llamada a la función sin parámetros
valida_opcion()

```

Resultado

Siguiendo las indicaciones del ejercicio se realizan las pruebas para comprobar su funcionamiento:

ios/EjerciciosResueltos/ud1_ejer2_validad_opcion_menu.py

```

=====
GAMIFICA EN EL AULA
=====
1 - Cargar datos del fichero
2 - Imprimir datos
3 - Jugar
4 - Guardar datos
5 - Cambiar contraseña
0 - Salir
Indica la opción seleccionada (0-5): hola
Error. El dato no es un entero, vuelve a intentarlo.
=====
GAMIFICA EN EL AULA
=====
1 - Cargar datos del fichero
2 - Imprimir datos
3 - Jugar
4 - Guardar datos
5 - Cambiar contraseña
0 - Salir
Indica la opción seleccionada (0-5): 1
La opción seleccionada es: 1

```

GAMIFICA EN EL AULA

```

=====
1 - Cargar datos del fichero
2 - Imprimir datos
3 - Jugar
4 - Guardar datos
5 - Cambiar contraseña
0 - Salir
Indica la opción seleccionada (0-5): 6
ERROR: 6 no es un nº entero entre 0 y 5, vuelve a intentarlo.
=====
GAMIFICA EN EL AULA
=====
1 - Cargar datos del fichero
2 - Imprimir datos
3 - Jugar
4 - Guardar datos
5 - Cambiar contraseña
0 - Salir
Indica la opción seleccionada (0-5): 0
Fin del programa, hasta pronto.

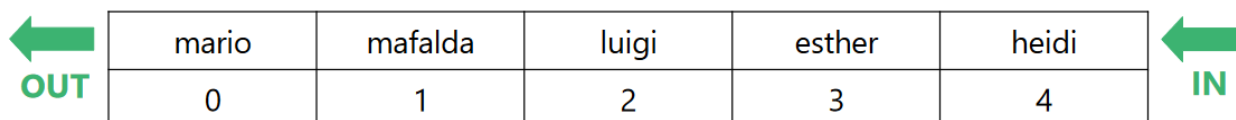
```

Se ha añadido en el código que los mensajes de error aparezcan de color rojo.

Actividad UD03

Vamos a crear un programa que gestione la lista de jugadores on-line de un juego.

Para simplificar su gestión simularemos una cola FIFO (First In First Out), de tal forma que, cuando llegue un jugador lo añadiremos al final de la cola y cuando se vaya un jugador siempre se irá el de la primera posición.



El programa debe funcionar de la siguiente forma:

- 1. Inicio:** podemos tener una lista creada de jugadores on-line con varios de ellos. Si es así, la mostraremos al principio del programa para que el usuario sepa con qué jugadores cuenta ya la lista.
- 2. Cuerpo:** mostraremos un menú como en el que se muestra a continuación.

```

=====
                JUGADORES ON-LINE
=====
1 - Llega un jugador nuevo
2 - Se va un jugador
3 - FIN
-----
Dame la opción:
  
```

Figura 4: Menú del programa

Según la opción seleccionada, el programa reaccionará como se indica en la siguiente tabla:

Opción	Respuesta
1	Se pide el nombre del nuevo jugador al usuario, se le da la bienvenida y se añade al final de la lista de jugadores. Además, se muestra la lista actual de jugadores.
2	Se extrae el jugador del principio de la lista y se muestra un mensaje informativo. Por ejemplo: "El jugador Pepito se va". Además, se muestra la lista actual de jugadores.
3	Se muestra la lista actual de jugadores y acaba el programa.
Otro	Se mostrará un mensaje de error y se seguirá pidiendo la opción.

- 3. Final:** mostraremos un mensaje de despedida.

A continuación puedes ver un ejemplo completo de ejecución del programa:

```

LISTA DE JUGADORES: mario - mafalda - luigi - esther - heidi - songoku - beauty - beast -

=====
JUGADORES ON-LINE
=====
1 - Llega un jugador nuevo
2 - Se va un jugador
3 - FIN
-----
Dame la opción: 1

¿Quién eres? pep
Bienvenid@ jugador pep
LISTA DE JUGADORES: mario - mafalda - luigi - esther - heidi - songoku - beauty - beast - pep -

=====
JUGADORES ON-LINE
=====
1 - Llega un jugador nuevo
2 - Se va un jugador
3 - FIN
-----
Dame la opción: 1

¿Quién eres? mireia
Bienvenid@ jugador mireia
LISTA DE JUGADORES: mario - mafalda - luigi - esther - heidi - songoku - beauty - beast - pep - mireia -

```

Figura 5: Añadimos dos jugadores nuevos

```

=====
JUGADORES ON-LINE
=====
1 - Llega un jugador nuevo
2 - Se va un jugador
3 - FIN
-----
Dame la opción: 2

Adiós al jugador mario

LISTA DE JUGADORES: mafalda - luigi - esther - heidi - songoku - beauty - beast - pep - mireia -

=====
JUGADORES ON-LINE
=====
1 - Llega un jugador nuevo
2 - Se va un jugador
3 - FIN
-----
Dame la opción: 2

Adiós al jugador mafalda

LISTA DE JUGADORES: luigi - esther - heidi - songoku - beauty - beast - pep - mireia -

```

Figura 6: Eliminamos dos jugadores

```

=====
JUGADORES ON-LINE
=====
1 - Llega un jugador nuevo
2 - Se va un jugador
3 - FIN
-----
Dame la opción: 3

LISTA DE JUGADORES: luigi - esther - heidi - songoku - beauty - beast - pep - mireia -
Hasta pronto

```

Figura 7: Salimos del programa

Puedes crear el programa desde cero o basarte en el esquema que tienes en el fichero `ud1_ejer3_jugadores_online (ESQUEMA).py` del aula virtual. En este caso, deberás sustituir las instrucciones `pass` por las instrucciones adecuadas para que funcione el programa según el enunciado.

A continuación se añade el código del ejercicio resuelto a partir de los ejercicios ejemplo de la unidad:

```
# Creando una función para imprimir listas
def imprimiendoLista(lista):
    #Imprimimos la lista de jugadores on-line
    print('lista de jugadores on-line'.upper())
    c = 1 # contador
    for elemento in lista:
        print(f'Jugador {c}: {elemento}', end=' - ')
        c+=1

#Creamos la lista con todos los jugadores on-line que hay conectados
jugadores = ["mario", "mafalda", "luigi", "esther", "heidi", "songoku", "beauty",
"beast"]

imprimiendoLista(jugadores)

acaba = False
while not acaba:
    #mostramos el menú y solicitamos la operación al usuario
    print('\n\u001B[0m=====') # Resetea colores
    print('\tJUGADORES ON-LINE')
    print('=====')
    print('1 - Llega un jugador nuevo')
    print('2 - Se va un jugador')
    print('3 - Fin')
    # bucle try-exception para controlar el mensaje de error cuando
    # el usuario no añada un entero
    try:
        opcion = int(input('Indica la opción seleccionada (1-3): '))

        #Respondemos a la opción seleccionada
        if opcion == 1:
            #preguntamos el nombre, damos la bienvenida y añadimos a la lista
            nom = input('¿Cómo te llamas? ')
            print(f'Bienvenido@ jugador {nom}')
            jugadores.append(nom)
            #Imprimiendo lista
            imprimiendoLista(jugadores)

        elif opcion == 2:
```

```
print(f'Adiós al jugador {jugadores[0]}')
jugadores.pop(0) # borrando el 1º elemento de la lista
#Imprimiendo lista
imprimiendoLista(jugadores)

elif opcion == 3:
    acaba = True
else:
    print(f'\u001B[31mERROR: {opcion} no es un nº entero entre 1 y 3, vuelve a intentarlo.')

except ValueError:
    print("\u001B[31mError. El dato no es un entero, vuelve a intentarlo.")

print("Hasta pronto")
print()
```

Resultado

El resultado de las pruebas es el siguiente:

```
LISTA DE JUGADORES ON-LINE
Jugador 1: mario - Jugador 2: mafalda - Jugador 3: luigi - Jugador 4: esther - Jugador 5: heidi - Jugador 6: songoku
- Jugador 7: beauty - Jugador 8: beast -

=====
JUGADORES ON-LINE
=====
1 - Llega un jugador nuevo
2 - Se va un jugador
3 - Fin
Indica la opción seleccionada (1-3): hola
Error. El dato no es un entero, vuelve a intentarlo.

=====
JUGADORES ON-LINE
=====
1 - Llega un jugador nuevo
2 - Se va un jugador
3 - Fin
Indica la opción seleccionada (1-3): 0
ERROR: 0 no es un nº entero entre 1 y 3, vuelve a intentarlo.

=====
JUGADORES ON-LINE
=====
```

```
=====
JUGADORES ON-LINE
=====
1 - Llega un jugador nuevo
2 - Se va un jugador
3 - Fin
Indica la opción seleccionada (1-3): 1
¿Cómo te llamas? pep
Bienvenido@ jugador pep

LISTA DE JUGADORES ON-LINE
Jugador 1: mario - Jugador 2: mafalda - Jugador 3: luigi - Jugador 4: esther - Jugador 5: heidi - Jugador 6: songoku
- Jugador 7: beauty - Jugador 8: beast - Jugador 9: pep -

=====
JUGADORES ON-LINE
=====
1 - Llega un jugador nuevo
2 - Se va un jugador
3 - Fin
Indica la opción seleccionada (1-3): 1
¿Cómo te llamas? maria
Bienvenido@ jugador maria

LISTA DE JUGADORES ON-LINE
Jugador 1: mario - Jugador 2: mafalda - Jugador 3: luigi - Jugador 4: esther - Jugador 5: heidi - Jugador 6: songoku
- Jugador 7: beauty - Jugador 8: beast - Jugador 9: pep - Jugador 10: maria -

=====
JUGADORES ON-LINE
=====
1 - Llega un jugador nuevo
2 - Se va un jugador
3 - Fin
Indica la opción seleccionada (1-3): 2
Adiós al jugador mario

LISTA DE JUGADORES ON-LINE
Jugador 1: mafalda - Jugador 2: luigi - Jugador 3: esther - Jugador 4: heidi - Jugador 5: songoku - Jugador 6: beauty
- Jugador 7: beast - Jugador 8: pep - Jugador 9: maria -

=====
JUGADORES ON-LINE
=====
1 - Llega un jugador nuevo
2 - Se va un jugador
3 - Fin
Indica la opción seleccionada (1-3): 2
Adiós al jugador mafalda

LISTA DE JUGADORES ON-LINE
Jugador 1: luigi - Jugador 2: esther - Jugador 3: heidi - Jugador 4: songoku - Jugador 5: beauty - Jugador 6: beast -
Jugador 7: pep - Jugador 8: maria -

=====
JUGADORES ON-LINE
=====
1 - Llega un jugador nuevo
2 - Se va un jugador
3 - Fin
Indica la opción seleccionada (1-3): 3
Hasta pronto
```

Se ha añadido en el código que los mensajes de error aparezcan de color rojo, aunque en las imágenes no aparezca ya que se realizaron antes de modificarlo.

Actividad UD04

Crear un programa que utilice un diccionario para gestionar la información de un vuelo. Originalmente el diccionario contendrá la siguiente información:

Clave	Valor
origen	valencia
destino	menorca
día	15-08
clase	turista

El programa mostrará el siguiente menú:

```

=====
      GESTIÓN DE UN VUELO
=====
1 - Imprimir datos vuelo
2 - Imprimir claves
3 - Añadir <pasajeros>
4 - Imprimir un valor
5 - Borrar clave
0 - SALIR
-----
Dame la opción:

```

Según la opción seleccionada, el programa reaccionará como se indica en la siguiente tabla:

Opción	Respuesta
1	Se imprimen los datos del diccionario (claves y valores).
2	Se imprimen las claves del diccionario como una lista de valores. Ejemplo: CLAVES: origen, destino, día, clase,
3	Se pide el número de pasajeros del vuelo y se crea la clave “pasajeros” con el valor introducido por el usuario.
4	Se pregunta al usuario el nombre de una clave del diccionario. Si es una clave correcta, se muestra el valor asociado. Si no, se avisa del error.
5	Se pregunta al usuario el nombre de la clave que quiere borrar del diccionario. Si es una clave correcta, se borra el par (clave:valor) del diccionario. Si no, se avisa del error.

0	El programa acaba.
---	--------------------

Después de realizar las tareas correspondientes a la opción seleccionada, se volverá a mostrar el menú al usuario.

Puedes crear el programa desde cero o basarte en el esquema que tienes en el fichero `ud1_ejer4_vuelo(ESQUEMA).py` del aula virtual. En este caso, deberás sustituir las instrucciones `pass` por las instrucciones adecuadas para que funcione el programa según el enunciado.

Como en los casos anteriores se añade el código del ejercicio, que es similar al anterior pero trabajando con diccionarios y con el esquema aportado por el profesor.

```
vuelo = {'origen':'Valencia', 'destino':'Menorca', 'día':'15-08', 'clase':'turista'}
acaba = False
while not acaba:
    #Mostramos el menú y solicitamos la operación al usuario
    opcion = ''
    while opcion not in ['1', '2', '3', '4', '5', '0']:
        print()
        print("\u001B[0m=====") # Resetea colores
        print("  GESTIÓN DE UN VUELO  ")
        print("=====")
        print(" 1 - Imprimir datos vuelo")
        print(" 2 - Imprimir claves ")
        print(" 3 - Añadir <pasajeros> ")
        print(" 4 - Imprimir un valor")
        print(" 5 - Borrar clave ")
        print(" 0 - SALIR ")
        print("-----")
        opcion = input("Dame la opción: ")
        print()

    #Respondemos a la opción seleccionada
    if opcion == '1': #Imprimir datos del diccionario
        print("  DATOS DEL VUELO:")
        print('  -----')
        for k,v in vuelo.items(): # se recorre keys y values
            print('%9s: %8s' %(k,v))
    elif opcion == '2': #Imprimir las claves del diccionario
        print("CLAVES: ", end='')
        for clave in vuelo.keys():
            print (clave, end=", ")
    elif opcion == '3': #Añadir la clave 'pasajeros' con su valor
        nPasajeros = input('Indica el número de pasajeros en el viaje: ')
        if nPasajeros.isdigit(): # otra manera de comprobar que es un entero
            vuelo['pasajeros'] = nPasajeros
        else:
```

```
    print(f'\u001B[31mERROR: {nPasajeros} no es un número entero.')
elif opcion == '4': #Dada una clave, imprimir un valor
    claveUser = input('Indica la clave del valor a mostrar: ')
    error = True
    for clave in vuelo.keys():
        if clave.lower() == claveUser.lower(): #para evitar case sensitive
            print(f'valor de {claveUser.lower()}: {vuelo[claveUser.lower()]}')
            error=False
            break
    if error:
        print(f'\u001B[31mERROR: {claveUser} no se encuentra dentro de'
              +' la información del vuelo.')

elif opcion == '5': #Borrar una clave con su valor
    claveUser = input('Indica la clave a borrar: ')
    error = True
    for clave in vuelo.keys():
        if clave.lower() == claveUser.lower(): #para evitar case sensitive
            print(f'la "clave: valor" a borrar es {claveUser.lower()}: '
                  +f' {vuelo[claveUser.lower()]}')
            vuelo.pop(claveUser.lower())
            error=False
            break # sale del bucle for y continua
    if error:
        print(f'\u001B[31mERROR: {claveUser} no se encuentra dentro de'
              +' la información del vuelo, luego no se puede borrar.')

else:
    acaba = True
print("Hasta pronto")
```

Resultado

Tras realizar pruebas del funcionamiento del código anterior se obtiene los siguientes resultados:

```

=====
GESTIÓN DE UN VUELO
=====
1 - Imprimir datos vuelo
2 - Imprimir claves
3 - Añadir <pasajeros>
4 - Imprimir un valor
5 - Borrar clave
0 - SALIR
-----
Dame la opción: hola
Sin información
=====
GESTIÓN DE UN VUELO
=====
1 - Imprimir datos vuelo
2 - Imprimir claves
3 - Añadir <pasajeros>
4 - Imprimir un valor
5 - Borrar clave
0 - SALIR
-----
Dame la opción: 7
Sin información
=====
GESTIÓN DE UN VUELO
=====
1 - Imprimir datos vuelo
2 - Imprimir claves
3 - Añadir <pasajeros>
4 - Imprimir un valor
5 - Borrar clave
0 - SALIR
-----
Dame la opción: 1
DATOS DEL VUELO:
-----
origen: Valencia
destino: Menorca
día: 15-08
clase: turista
pasajeros: 3
=====
1 - Imprimir datos vuelo
2 - Imprimir claves
3 - Añadir <pasajeros>
4 - Imprimir un valor
5 - Borrar clave
0 - SALIR
-----
Dame la opción: 5
Indica la clave a borrar: pasaje
ERROR: pasaje no se encuentra dentro de la información del vuelo, luego no se puede borrar.
=====
GESTIÓN DE UN VUELO
=====
1 - Imprimir datos vuelo
2 - Imprimir claves
3 - Añadir <pasajeros>
4 - Imprimir un valor
5 - Borrar clave
0 - SALIR
-----
Dame la opción: 2
CLAVES: origen, destino, día, clase,
=====
GESTIÓN DE UN VUELO
=====
1 - Imprimir datos vuelo
2 - Imprimir claves
3 - Añadir <pasajeros>
4 - Imprimir un valor
5 - Borrar clave
0 - SALIR
-----
Dame la opción: 3
Indica el número de pasajeros en el viaje: 3
=====
1 - Imprimir datos vuelo
2 - Imprimir claves
3 - Añadir <pasajeros>
4 - Imprimir un valor
5 - Borrar clave
0 - SALIR
-----
Dame la opción: 4
Indica la clave del valor a mostrar: pasajeros
valor de pasajeros: 3
=====
GESTIÓN DE UN VUELO
=====
1 - Imprimir datos vuelo
2 - Imprimir claves
3 - Añadir <pasajeros>
4 - Imprimir un valor
5 - Borrar clave
0 - SALIR
-----
Dame la opción: 1
DATOS DEL VUELO:
-----
origen: Valencia
destino: Menorca
día: 15-08
clase: turista
pasajeros: 3
=====
1 - Imprimir datos vuelo
2 - Imprimir claves
3 - Añadir <pasajeros>
4 - Imprimir un valor
5 - Borrar clave
0 - SALIR
-----
Dame la opción: 5
Indica la clave a borrar: pasaje
ERROR: pasaje no se encuentra dentro de la información del vuelo.
=====
GESTIÓN DE UN VUELO
=====
1 - Imprimir datos vuelo
2 - Imprimir claves
3 - Añadir <pasajeros>
4 - Imprimir un valor
5 - Borrar clave
0 - SALIR
-----
Dame la opción: 4
Indica la clave del valor a mostrar: pasaje
la "clave: valor" a borrar es pasajeros: 3
=====
GESTIÓN DE UN VUELO
=====
1 - Imprimir datos vuelo
2 - Imprimir claves
3 - Añadir <pasajeros>
4 - Imprimir un valor
5 - Borrar clave
0 - SALIR
-----
Dame la opción: 1
DATOS DEL VUELO:
-----
origen: Valencia
destino: Menorca
día: 15-08
clase: turista
=====
GESTIÓN DE UN VUELO
=====
1 - Imprimir datos vuelo
2 - Imprimir claves
3 - Añadir <pasajeros>
4 - Imprimir un valor
5 - Borrar clave
0 - SALIR
-----
Dame la opción: 0
Hasta pronto

```

Como se puede ver en la imagen anterior, se va utilizando las funciones a partir de las opciones, de arriba a abajo y de izquierda a derecha, hasta trabajar los máximos casos posibles indicados en la actividad. Por otro lado, se ha añadido en el código que los mensajes de error aparezcan de color rojo, aunque en las imágenes no aparezca ya que se realizaron antes de modificarlo.

Actividad Obligatoria

Vamos a crear un programa llamado `ud1_ejercicio_obligatorio.py` que gestione los datos de la lista de vuelos del Aeropuerto de Valencia.

Cada vuelo dispondrá de la siguiente información: origen, destino, día y clase e, inicialmente, ya se dispondrá de la información de los siguientes vuelos:

origen	destino	día	clase
Valencia	Menorca	15-08	turista
Valencia	Tenerife	20-08	turista
París	Valencia	15-08	primera
Atenas	Valencia	20-08	primera

El programa contará con una lista de diccionarios para almacenar esta información. Cada diccionario se corresponderá con un vuelo que almacenará pares clave-valor tal y como se indica en la tabla anterior.

El programa mostrará el siguiente menú:

```
=====
LISTA DE VUELOS
=====
1 - Imprimir datos vuelos
2 - Buscar por origen
3 - Imprimir un vuelo
4 - Cambiar fecha de vuelo
0 - SALIR
-----
Dame la opción:
```

Según la opción seleccionada, el programa ejecutará una función nueva creada por el programador, tal y como se explica en la siguiente tabla:

Opción	Función
1	<code>imprimir()</code> Se imprimirán los datos de todos los vuelos de la lista. La función recibirá la lista de vuelos y no devolverá nada.
2	<code>buscar_origen()</code> Se pedirá al usuario el origen de un vuelo y se mostrarán sus datos por

	pantalla. Si el vuelo no existiese, se mostraría un mensaje al usuario. La función recibirá la lista de vuelos y no devolverá nada.
3	<i>imprimir_vuelo()</i> Se pedirá al usuario el origen y destino de un vuelo y se imprimirá el vuelo correspondiente. Si el vuelo no existiese, se mostraría un mensaje al usuario. La función recibirá la lista de vuelos y no devolverá nada.
4	<i>cambiar_fecha()</i> Se pedirá al usuario el origen y destino de un vuelo. Si el vuelo no existiese, se mostraría un mensaje al usuario. Si existiese, se solicitaría una nueva fecha para el vuelo y se modificaría en el diccionario correspondiente de la lista. La función recibirá la lista de vuelos original y devolverá la lista de vuelos modificada.
0	El programa acaba.

Después de realizar las tareas correspondientes a la opción seleccionada, se volverá a mostrar el menú al usuario, excepto en la opción 0.

Puedes crear el programa desde cero o basarte en el esquema que tienes en el fichero `ud1_ejercicio_obligatorio (ESQUEMA).py` del aula virtual. En este caso, deberás sustituir las instrucciones `pass` por las instrucciones adecuadas para que funcione el programa según el enunciado.

Siguiendo todos los conceptos de la unidad, se procede a trabajarlos en el código siguiente del ejercicio, siguiendo además las indicaciones del ejercicio con el fichero que contiene el esquema:

```
vuelos = [{ 'origen': 'Valencia', 'destino': 'Menorca', 'día': '15-08', 'clase': 'turista' },
           { 'origen': 'Valencia', 'destino': 'Tenerife', 'día': '20-08', 'clase': 'turista' },
           { 'origen': 'París', 'destino': 'Valencia', 'día': '15-08', 'clase': 'primera' },
           { 'origen': 'Atenas', 'destino': 'Valencia', 'día': '20-08', 'clase': 'primera' } ]

# DEFINICIÓN DE FUNCIONES -----
def valida_opcion():
    '''Función que muestra un menú y valida que la opción sea correcta'''

    #Mostramos el menú y solicitamos la operación al usuario
    opciones_validas = ['1', '2', '3', '4', '0'] # Elimino la opción 5 ya que no aparece
    opcion = ''
    while (opcion not in opciones_validas):
        print()
        print("\u001B[0m=====") # Resetea colores
```

```
print("      LISTA DE VUELOS      ")
print("=====")
print(" 1 - Imprimir datos vuelos")
print(" 2 - Buscar por origen ")
print(" 3 - Imprimir un vuelo")
print(" 4 - Cambiar fecha de vuelo ")
print(" 0 - SALIR ")
print("-----")
opcion = input("Dame la opción: ")
if (opcion not in opciones_validas):
    print("Por favor, vuelve a intentarlo.")
else:
    print()
return opcion

def imprimir(Lista):
    '''Función que imprime la lista de vuelos.

    Parámetros de entrada: lista de diccionarios
    Parámetros de salida: no hay
    '''
    print("IMPRIMIR VUELOS:",end=' ')
    c = 0 # contador
    for vuelo in lista:
        c+=1
        print(f'\nvuelo {c} ', end='=> ')
        for k,v in vuelo.items():
            print('%8s: %8s' %(k,v), end= ',')

def buscar_origen(lista):
    '''Función que busca un vuelo dentro de una lista de vuelos.

    Parámetros de entrada: lista de diccionarios
    Parámetros de salida: no hay
    '''
    print("BUSCAR ORIGEN:",end=' ')
    c = 0 # contador
    origenVuelo = input('Indica el origen del vuelo a buscar: ')
    # doble b́ucle ćclico para obtener los datos de los objetos de la lista
    for vuelo in lista:
        if vuelo['origen'].lower() == origenVuelo.lower().strip():
            c+=1
            print(f'\nvuelo {c} ', end='=> ')
            #imprimiendo los datos del vuelo
            for k,v in vuelo.items():
                print('%8s: %8s' %(k,v), end= ',')
```

```
#break # si solo queremos que encuentre el primero
if c == 0:
    print(f'\u001B[31mERROR: No hay vuelo con origen en: {origenVuelo}\n')
    print()

def imprimir_vuelo(lista):
    '''Función que imprime los datos de un vuelo.

    Parámetros de entrada: lista de diccionarios
    Parámetros de salida: no hay
    '''
    print("IMPRIMIR VUELO:")
    c = 0 # contador
    origen = input('Indica el origen del vuelo: ').lower().strip()
    destino = input('Indica el destino del vuelo: ').lower().strip()
    for vuelo in lista:
        if vuelo['origen'].lower() == origen and vuelo['destino'].lower() == destino:
            c+=1
            print(f'\n- Vuelo {c}.')
            #imprimiendo los datos del vuelo
            for k,v in vuelo.items():
                print('%8s: %8s' %(k,v))
            #break # si solo queremos que encuentre el primero
    if c == 0:
        print(f'\u001B[31mERROR: No hay vuelo con origen "{origen}" y destino'
            +f' "{destino}". Revise su vuelo y verifique los datos, gracias.')

def cambiar_fecha(lista):
    '''Función que cambia la fecha de un vuelo.

    Parámetros de entrada: lista de diccionarios
    Parámetros de salida: lista de diccionarios actualizada
    '''
    print("CAMBIAR FECHA DE VUELO:")
    c = 0 # contador
    nuevaFecha = ''
    orig = input('Indica el origen del vuelo: ').lower().strip()
    dest = input('Indica el destino del vuelo: ').lower().strip()
    for vuelo in lista:
        if vuelo['origen'].lower() == orig and vuelo['destino'].lower() == dest:
            c+=1
            nuevaFecha = input(f"El vuelo de {orig} a {dest} saldrá el día {vuelo['día']}"
                +f". Indique el cambio de fecha (dd-mm o ENTER"
                +" para salir): ").strip()
            if nuevaFecha: # en caso de añadir enter, no modifica los datos
                #No hay protección en los datos del cambio de día
```

```
        vuelo['día'] = nuevaFecha
        print(f'\n- IMPRIMIENDO VUELO MODIFICADO.')
    else:
        print(f'\n- IMPRIMIENDO VUELO SIN CAMBIOS.')
    #imprimiendo los datos del vuelo
    for k,v in vuelo.items():
        print('%8s: %8s' %(k,v))
    break # si solo queremos que encuentre el primero y lo modifique
if c == 0:
    print(f'\u001B[31mERROR: No hay vuelo con origen "{orig}" y destino'
          +f' "{dest}". Revise su vuelo y verifique los datos, gracias.')
return lista # obligatorio si no queremos recibir un None

# PROGRAMA PRINCIPAL -----
opcion = valida_opcion()
while opcion != '0':
    #Respondemos a la opción seleccionada
    if opcion == '1':
        imprimir(vuelos)
    elif opcion == '2':
        buscar_origen(vuelos)
    elif opcion == '3':
        imprimir_vuelo(vuelos)
    elif opcion == '4':
        vuelos = cambiar_fecha(vuelos)
    opcion = valida_opcion()

print("Hasta pronto")
```

Resultado

Tras realizar pruebas del funcionamiento del código anterior se obtiene los siguientes resultados:

[illegible]

Como se puede ver en la imagen anterior, se va utilizando las funciones a partir de las opciones, de arriba a abajo y de izquierda a derecha, hasta trabajar los máximos casos posibles indicados en la actividad. Por otro lado, se ha añadido en el código que los mensajes de error aparezcan de color rojo, aunque en las imágenes no aparezca ya que se realizaron antes de modificarlo.

Problemas encontrados, soluciones y/o sugerencias

No se han encontrado problemas que dificulten la realización de nuevas pruebas o aspectos que solucionar ya que las actividades estaban muy bien redactadas y con pruebas de ejemplo para poder simular con nuestro código

Conclusión

Unos casos prácticos muy útiles donde trabajar los conceptos introductorios de la unidad donde se aplica la información de los ejercicios de ejemplo de una manera muy didáctica y fluida con el objetivo de utilizarlo en la actividad obligatoria lo que reduce la frustración y mejora el aprendizaje.

Bibliografía/webgrafía

- Documentación del curso.
- Repositorio de GitHub compartido: <https://github.com/oscgaresc/ADA>